Problem Set 1

Graded

Student

Jinku Jung

Total Points

32 / 40 pts

Question 1

1.10 **5** / 5 pts

- ✔ **+ 1 pt** a) 1st iteration correct
- ✔ **+ 1 pt** a) 2nd iteration correct
- ✔ **+ 1 pt** a) 3rd iteration correct
- ✔ **+ 1 pt** b) Correct explanation of divide by zero error
- ✔ **+ 1 pt** b) Correct modification to the algorithm

Question 2

1.11 **3** / 5 pts

- ✔ **+ 1 pt** Calculation number of paths correct
- ✔ **+ 1 pt** Time calculation correct
- ✔ **+ 1 pt** Identifies and explains why algorithm is not feasible
- 💬 (+3/5)
  Alternate algorithm was not provided

Question 3

2.20 **2** / 5 pts

- ✔ **+ 2 pts** Unambiguous language and structure (a paragraph is ok as long as it is clear what steps are taken)
- 💬 your algorithm is going to print that, for example 6 is a prime number, because when you compute result for index = 5, then 6 mod 5 is not 0, and so the code will enter the else if statement; the cases N=1, N=2 need to be handled separately

Question 4

2.23 **4** / 5 pts

- ✔ **+ 2 pts** Unambiguous language and structure (a paragraph is ok as long as it is clear what steps are taken)
- 💬 **+ 2 pts** Algorithm should not increment j++ twice

Question 5

2.25 — **5** / 5 pts

- ✔ **+ 2 pts** Correct Logic
- ✔ **+ 1 pt** Does not access length of list or indexes
- ✔ **+ 2 pts** Unambiguous language and structure (a paragraph is ok as long as it is clear what steps are taken)
- 💬 please tag the correct page!

Question 6

3.11 — **4** / 5 pts

- ✔ **+ 1 pt** a) Correct exchanges
- ✔ **+ 1 pt** b) Correct exchanges
- ✔ **+ 1 pt** c) Correct exchanges
- ✔ **+ 1 pt** Shows work (even if incorrect)

Question 7

3.26 — **5** / 5 pts

- ✔ **+ 5 pts** Correct
- 💬 Good job! Next time for pseudocode, please write the output of your program so we can verify it is correct a bit faster.

Question 8

3.33 **Resolved** 4 / 5 pts

✔ **+ 2 pts** Correct number of searches

✔ **+ 2 pts** Provides rationale and sets up equation correctly

✔ **+ 1 pt** Shows calculations (can get this point even if they are incorrect)

✔ **− 1 pt** Correct, but uses n^2 instead of n(n-1)/2

Regrade Request **Submitted on: Feb 05**

Dear Grader:

I had the "(n * (n - 1)) / 2)" expression stated in my T(n) time-complexity function for Selection Sort, which I simplify in the very next line to a Big-O approximation in an O(n^2) function for large 'n' for Selection Sort. Note: the grading rubric allows a Big-O simplification for large 'n' for the Binary Search algorithm, so why is using the same technique wrong for Selection Sort? This does not affect my final answer since I do get credit for the correct number of searches. I was just told in my Discrete Math class and textbook (Discrete Math and its Applications; Epp, 4th edition) that to go from T(n) to O(n) format, you had to account for cases when 'n" gets very large and thus pick the highest order term. Here, I thought that actual time-complexity and Big-O complexity were two separate concepts. Please consider.

Respectfully submitted,

Jinku Jung

The question asked you to calculate when the selection sort + binary search would become more efficient in terms of # of comparisons. This last part is important - we need to use the # of comparisons formula rather than the Big-O formula. Saying that there are n^2 comparisons in selection sort would be incorrect. We would have preferred you to also use the exact formula for binary search as well, but we realized that the majority of students weren't aware of it so we decided not to penalize student for that.

Reviewed on: Feb 08

Question assigned to the following page: 1

Problem Set #1

Jiakun Jung
COMS 1004
Spring 2021
Section 2
1/20/2021
UNI: dj2598

Ch. V #10 (1) Stepping through Euclid's GCD algorithm

① i [32] j [20] R [not set]

② R of $\left(\frac{i}{j}\right)$? → R of $\frac{32}{20}$? → R [12]

③ i [20] j [12]

↓ (repeating, ②)

R of $\left(\frac{i}{j}\right)$? = R of $\frac{20}{12}$? → R [8]

↓ (repeat ③)

i [12] j [8]

↓ (repeat ②)

R of $\left(\frac{i}{j}\right)$? = R of $\frac{12}{8}$? → R [4]

↓ (repeat ③)

i [8] j [4]

↓ (repeat ②)

R of $\left(\frac{i}{j}\right)$ = R of $\frac{8}{4}$? → R [∅]

④ Output: j [4] + "is your GCD"

⑤ BREAK

(b.) Given inputs '∅' and "32"?

* Create Step (1.5): If j = ∅, Output: "ERROR! j = ∅" causes a divide by zero error in this algorithm! BREAK

Question assigned to the following page: 2

Pg 219 Ch. 1 #11

Total # of paths possible?

$25!$ paths total

$\cong 1.5 \times 10^{25}$ unique paths possible

Computer's rate of calculation?

1.0×10^{7} $\frac{\text{paths evaluated}}{\text{second}}$

Total computing time needed to find the optimal path?

$$\text{Total\_Computing\_Time} = \frac{\text{Total\_Paths\_Possible}}{\text{Rate\_Of\_Calculation}}$$

$$\cong \frac{1.5 \times 10^{25} \text{ paths}}{1.0 \times 10^{7} \frac{\text{paths}}{\text{second}}}$$

$$\cong (1.5 \times 10^{18} \text{ seconds}) * \left(\frac{1 \text{ hour}}{3600 \text{ seconds}}\right) * \left(\frac{1 \text{ day}}{24 \text{ hours}}\right) * \left(\frac{1 \text{ year}}{365 \text{ days}}\right) * \left(\frac{1 \text{ century}}{100 \text{ years}}\right)$$

$\cong 4.76 \times 10^{8}$ centuries

* This is a terrible algorithm to use to solve the Traveling Salesman problem!

* $O(n!)$ is the worst time complexity an algorithm can have; you would probably get kicked out of your local UNIX cluster if you ran this algorithm on a public workstation!!!

Question assigned to the following page: 3

[Pg 3/9] Ch. 2/#20

Write your own "isPrime" algorithm, given a positive integer 'N'. Output either "N is prime" OR "N is NOT prime: number 'X' is a factor of 'N'."

① Ask user for their value of N

② Initialize Index = 2.

③ Initialize Result = 1

④ While (index < N && Result != 0) {

⑤ Result = (N mod index);

⑥ Index++;

}

⑦ If (Result = 0) {
println (N + " is NOT prime.");

} Else if (Result != 0)
println (N + " is a prime number.");

Ch 2/#23

① Input & store all values from $N_1, \ldots N_k$ AND the value for SUM.

② Initialize i to 1.

③ Initialize j to 2.

④ While (i < k) {

⑤ While (j ≤ k) {

⑥ If ($N_i + N_j$ == SUM) {

⑦ Print("N_i & N_j add up to the value of SUM")

⑧ BREAK

} Else

⑨ j++;

Question assigned to the following page: 4

Pg 4/9

⑤

⑩ i++;

⑪ j++;

⑫ Println("ERROR: No two numbers in this set add up to the value of SUM.");

Ch. 2/#25

① Initialize bool Adjacent = FALSE;

② Initialized Grab & Store integer values for V_1 & V_2

③ While((V_2 != -1) && (Adjacent == "FALSE")) {

If (v_1 == V_2) {

adjacent = "TRUE";

} Else {

V_1 = V_2;

V_2 = GetLine(v_next);

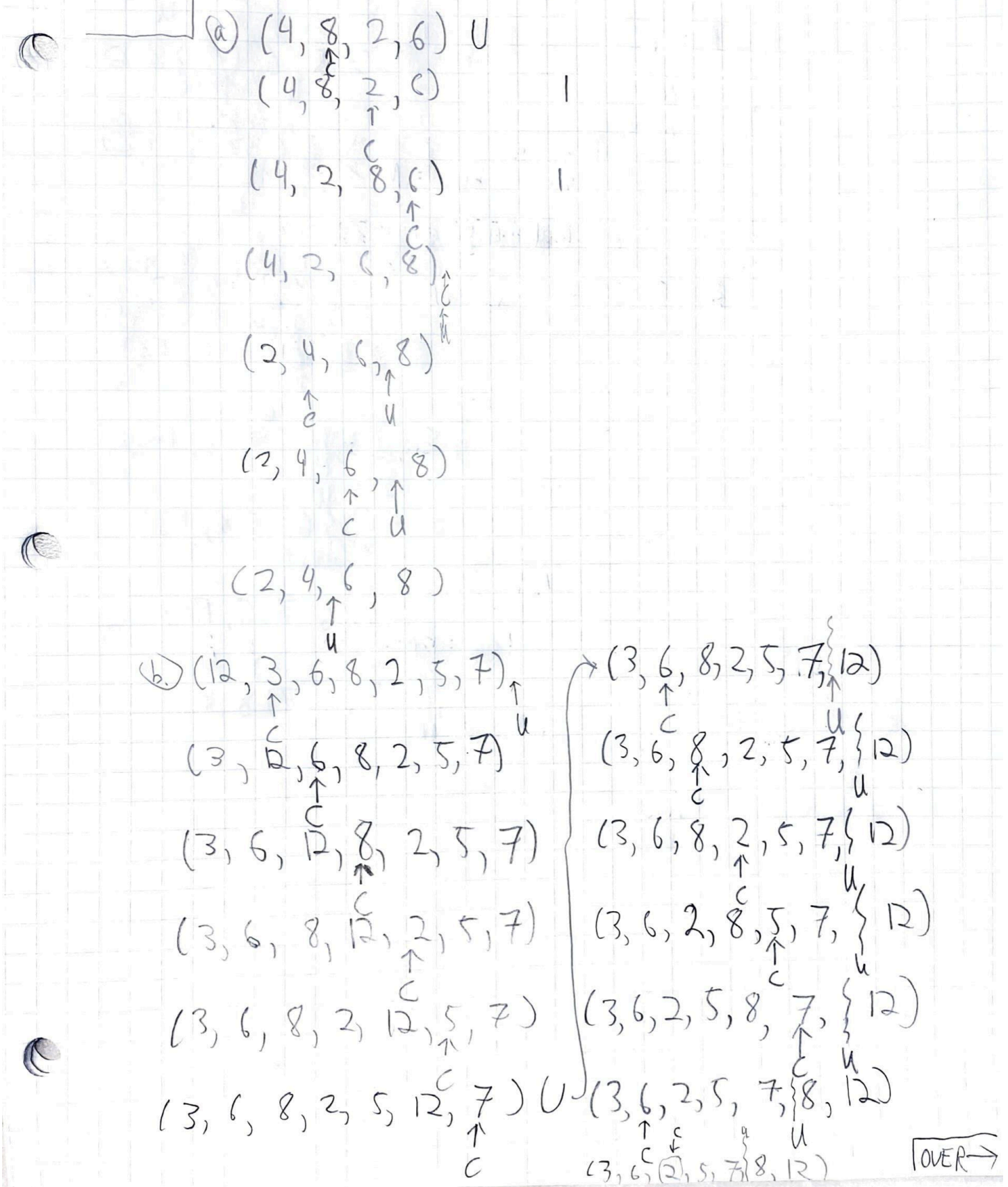
}

④ Println(Adjacent);

⑤ BREAK

Question assigned to the following page: 5

Pg 5/9 | Ch. 3 / #11 | Apply BubbleSort to these lists of values:

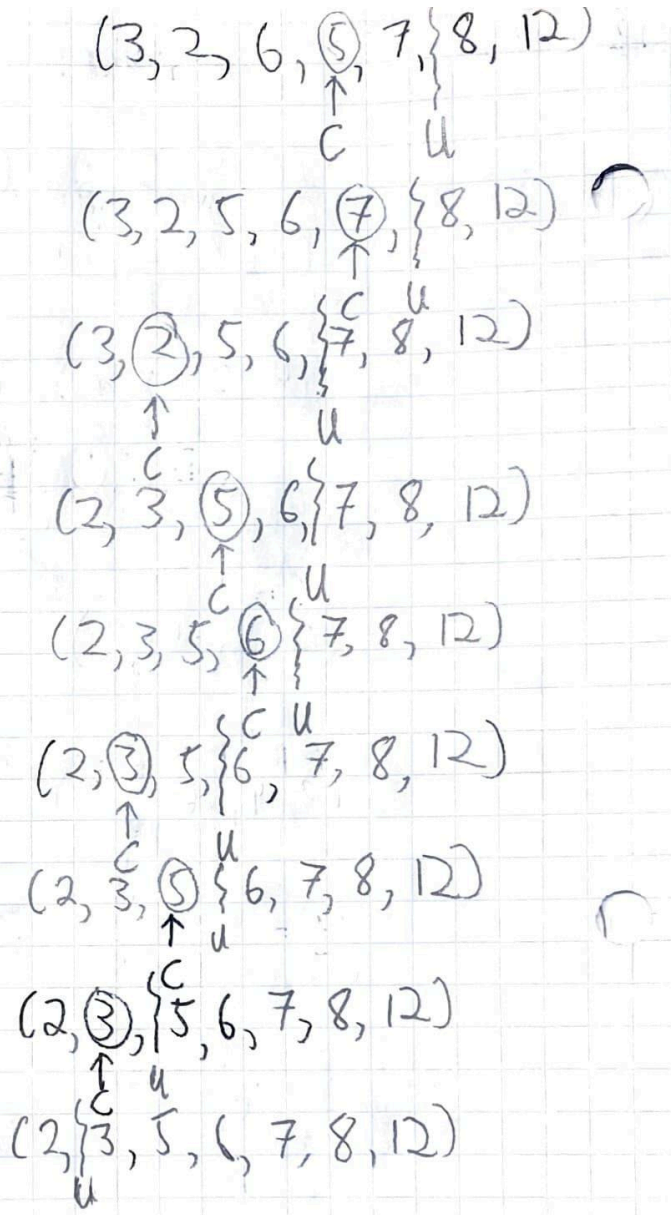


OVER→

Question assigned to the following page: 6

Pg 619

(cont'd)



END

Question assigned to the following page: 7

Pg 7/9 <#11 cont'd> ⓒ (D, (B), G, F, A, C, E, H){

(B, D, (G), F, A, C, E H){

(B, D, G, (F), A, C, E, H){

(B, D, F, G, (A), C, E, H){

(B, D, F, A, G, (C), E, H){

(B, D, F, A, C, G, (E), H){

(B, D, F, A, C, E, G, (H)){

(B, (D), F, A, C, E, G, {H)

(B, D, (F), A, C, E, G, {H)

(B, D, F, (A), C, E, G, {H)

(B, D, A, F, (C), E, G, {H)

(B, D, A, C, F, (E), G, {H)

END <= (A, {B, C, D, E, F, G, H) <= (A, (B), C, D, E, F, G, H)

(B, D, A, C, E, F, (G), {H)

(B, (D), A, C, E, F, {G, H)

(B, D, (A), C, E, F, {G, H)

(B, A, D, (C), E, F, {G, H)

(B, A, C, D, (E), F, {G, H)

(B, A, C, D, E, (F), {G, H)

(B, (A), C, D, E, {F, G, H)

(A, B, (C), D, E, {F, G, H)

(A, B, C, (D), E, {F, G, H)

(A, B, C, D, (E), F, G, H)

(A, (B), C, D, {E, F, G, H)

(A, B, (C), D, {E, F, G, H)

(A, B, C, (D), {E, F, G, H)

(A, (B), C, {D, E, F, G, H)

(A, B, (C), {D, E, F, G, H)

(A, (B), C, D, E, F, G, H)

Question assigned to the following page: 8

Pg 8/9

Ch. 3/#26

Use BST algorithm to check if "35" is in the given list:

A = (3, 6, 7, 9, 12, 14, 18, 21, 22, 31, 43)

① Begin = 1; End = 11; M; bool Found = "FALSE"; Scope;
What We Want: 35
What We Have: 14
Want > Have → Begin = M + 1;

② Begin 7 End 11 A(M) 22

A = (3, 6, 7, 9, 12, 14, 18, 21, 22, 31, 43)
Scope

What We Want: 35
What We Have: 22
Want > Have → Begin = M + 1;

③ Begin 9 End 11 A(M) 31

What We Want: 35
What We Have: 31
Want < Have →
End = m − 1;

$$M = \left[\frac{(End - Begin)}{2}\right] + Begin$$

$$= 1 + 9 = 10$$

④ Begin 9 End 9 A(M) 22

What We Want: 35
What We Have: 22
Want > Have → Begin = M + 1;
BUT: ① Begin = 10 while End = 9
② Begin > End
③ "while (Found == "FALSE" &&
Begin ≤ End)"
loop test condition is now FALSE!
BREAK (out of while loop)
④ Found FALSE →
println ("ERROR: " + Want +
"is NOT on the list");
⑤ END

No questions assigned to the following page.

Pg 9/9 Ch. 3 #33

(where "s" = # of searches done)

Case 1: seq search on unsorted list? $O(n)$ (worst case) = 11,000 * s

Case 2: Selection sort $\Rightarrow$ BST algorithm for searching the sorted list?

$$T(n) = \frac{2(n-1)}{2} + \left(1 + \lg_2(n)\right)$$ (where (...) assumes a perfectly sorted list)

$$T(n) \approx n^2 + \left[\left(1 + \lfloor \lg_2(n) \rfloor\right) * 5\right]$$

So

$$11{,}000s > (11{,}000)^2 + \left(1 + \lfloor \lg_2(11{,}000) \rfloor\right) * 5$$

$$11{,}000s > (11{,}000)^2 + 14 * 5$$

$$10{,}986s > (11{,}000)^2$$

$$s > \frac{(11{,}000)^2}{10{,}986}$$

$s > 11{,}014$ searches executed

for "Case 2" to be more algorithmically efficient than "Case 1." QED